

Lab Physical Storage & Indexing Strategies.

Name: Lê Hồng Anh

Student ID: 11247126

Lab 1: Inspecting Table Size

1, Goal: To inspect the physical storage consumption of tables and identify which tables in the `dvdrental` database are "Index Heavy" by comparing their data size against their index size.

2, Result:

QueryQuery History

1

SELECT

2

relname AS object_name,

3

pg_size_pretty(pg_table_size(fed.oid)) AS data_size,

4

pg_size_pretty(pg_indexes_size(fed.oid)) AS index_size,

5

pg_size_pretty(pg_total_relation_size(fed.oid)) AS total_size

Data OutputMessagesNotifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	object_name name	data_size text	index_size text	total_size text
1	rental	1232 kB	1120 kB	2352 kB
2	payment	896 kB	920 kB	1816 kB
3	film	736 kB	208 kB	944 kB
4	film_actor	272 kB	216 kB	488 kB
5	inventory	232 kB	208 kB	440 kB
6	customer	96 kB	112 kB	208 kB
7	address	88 kB	64 kB	152 kB
8	film_category	72 kB	40 kB	112 kB
9	city	64 kB	48 kB	112 kB
10	actor	40 kB	32 kB	72 kB
11	store	8192 bytes	32 kB	40 kB
12	staff	16 kB	16 kB	32 kB
13	language	8192 bytes	16 kB	24 kB
14	country	8192 bytes	16 kB	24 kB
15	category	8192 bytes	16 kB	24 kB
16	secure_orders	8192 bytes	16 kB	24 kB

Total rows: 16

Query complete 00:00:00.667

3, Discussion:

Looking at the output, we can identify several clearly "Index Heavy" tables:

- **The payment table:** This is a prime example. The physical data size is 896 kB , but the index size actually exceeds it at 920 kB .
- **The customer table:** Similarly, the index size (112 kB) is larger than the actual data stored in the table (96 kB).

- **The rental table:** While the data size is larger (1232 kB), the index size is extremely close (1120 kB), showing that nearly half of the total storage for this table is dedicated entirely to indexes.
- **Small reference tables:** Tables like store , language , country , and category all have a data size of just 8192 bytes (which is exactly 1 Page, 8KB), but their index sizes range from 16 kB to 32 kB

Lab 2: The Impact of a New Index

1. Goal: To measure the physical storage impact of adding a new index by calculating the exact number of 8KB Pages consumed by the newly created index.

2. Result

Before creating the new index:

The screenshot shows a query tool interface with a dark theme. The 'Query' tab is active, displaying a single SQL query: `SELECT pg_size_pretty(pg_indexes_size('payment'));`. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is selected, showing a table with one row. The table has a column header 'pg_size_pretty' with a lock icon and a data type 'text'. The row contains the value '920 kB'.

	pg_size_pretty text
1	920 kB

After creating the new index:

The screenshot shows the same query tool interface. The 'Query' tab now displays three SQL queries: `CREATE INDEX idx_payment_date_long ON payment (payment_date, amount, customer_id);`, an empty line, and `SELECT pg_size_pretty(pg_indexes_size('payment'));`. The 'Data Output' tab is selected, showing a table with one row. The table has a column header 'pg_size_pretty' with a lock icon and a data type 'text'. The row contains the value '1488 kB'. The text 'Showing rows: 1 to 1' is visible on the right side of the interface.

	pg_size_pretty text
1	1488 kB

3. Discussion

- **Initial index size:** 920 kB
- **Total index size after creation:** 1488 kB
- **Size of the new index:** 1488 kB - 920 kB = 568 kB
- **Pages Calculation:**
 - 568 kB / 8 kB = 71 Pages

Lab 3: Partial Index Practice

1. Goal To demonstrate the storage efficiency of a Partial Index by comparing its physical size to a hypothetical Full Index on the same column

2, Result:

Query

Query History

```

1 CREATE INDEX IF NOT EXISTS idx_expensive_films
2 ON film (replacement_cost)
3 WHERE replacement_cost > 25.00;
4
5 CREATE INDEX IF NOT EXISTS idx_full_replacement_cost
6 ON film (replacement_cost);
7
8 SELECT
9     indexrelname AS index_name,
10     pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
11 FROM pg_stat_user_indexes
12 WHERE indexrelname IN ('idx_expensive_films', 'idx_full_replacement_cost');

```

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📶

SQL

Showing

	index_name name	index_size text
1	idx_expensive_films	16 kB
2	idx_full_replacement_c...	40 kB

3, Discussion

- **Partial Index size:** 16 kB
- **Full Index size:** 40 kB

- **Analysis:** The partial index (`idx_expensive_films`) is significantly smaller (less than half the size) because it uses a `WHERE` clause to only store pointers for films with a `replacement_cost > 25.00` . The full index must store a B-Tree entry for every single row in the table.